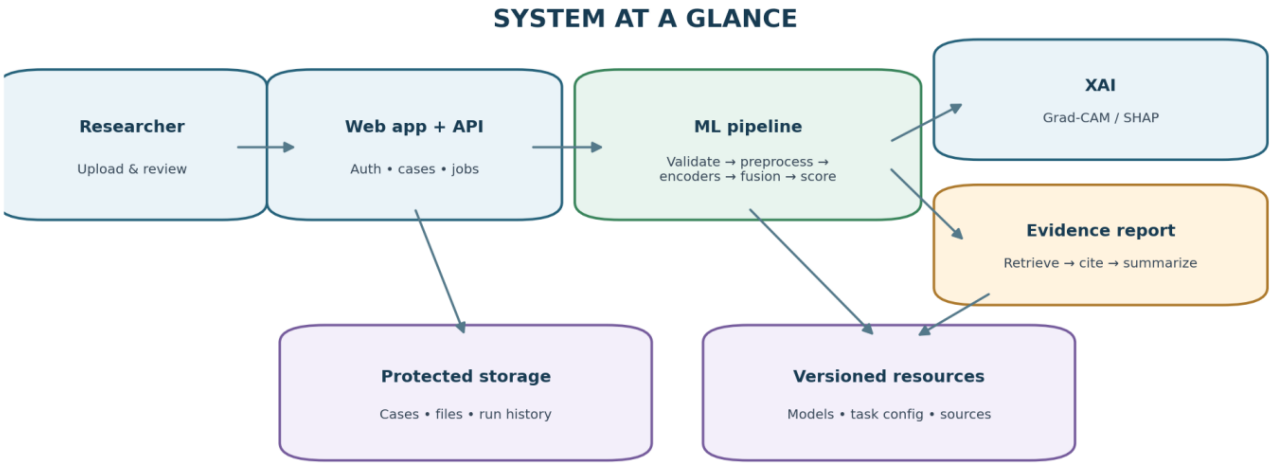
PROJECT SRS

A practical, visual guide for building the research prototype

Multimodal Medical Decision-Support System
Simplified developer edition • v1.0 • 13 October 2026



SAFETY BOUNDARY: Research prototype only—not a diagnosis or treatment tool.

Quick navigation

In Word, right-click here and choose Update Field to build the clickable contents.

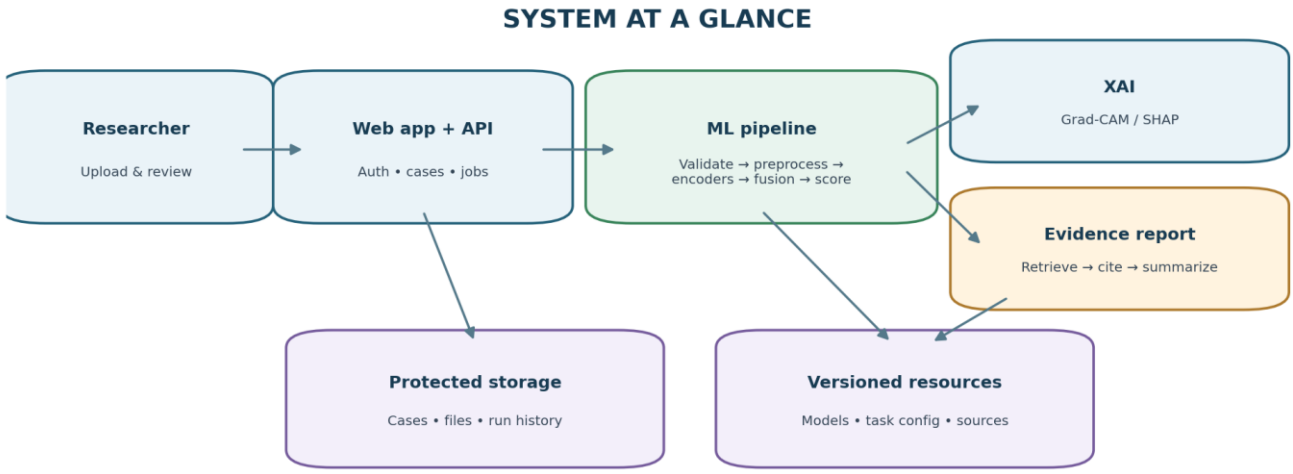
Use the checkboxes in “Client decisions” to close open choices. The diagrams are conceptual; exact technology choices remain configurable.

1. What are we building?

A web application for researchers to upload permitted, de-identified health data; run a configured AI model; inspect its prediction and explanations; and optionally create a summary linked to retrieved medical sources. A human reviews every output.

First release	Later extensions
One clearly defined dataset and prediction task	More tasks/datasets after the first task is tested
One or more modalities actually present in that dataset	Image + signal + structured + text only when paired data/model support them
Prediction + appropriate explanation	Curated RAG report, if approved source corpus and model are available
Research demo and reproducible evaluation	Hospital/clinical use is not included

2. How the parts fit together



Suggested implementation: modular monolith first, with clear boundaries for the web client, API, ML pipeline, XAI, retrieval/reporting and storage. Split into services only when deployment needs justify it.

3. Typical user journey



A run progresses through queued → validating → preprocessing → inference → optional XAI/RAG → succeeded, partial, or failed. An explanation/report failure must not hide a successful prediction; mark the run PARTIAL and explain what failed.

4. Product requirements (build checklist)

ID	The system shall…	Priority	How to check
R-01	Authenticate users and check permissions on every case, file, run, report and download.	Must	Try access as another user.
R-02	Create pseudonymous cases and accept only file types and fields allowed by the selected task.	Must	Valid + invalid upload tests.
R-03	Validate file contents, size, dimensions/schema and required metadata before inference.	Must	Corrupt, oversized, wrong-schema tests.
R-04	Apply the preprocessing configuration belonging to the selected model version.	Must	Fixture and repeatability tests.
R-05	Show which modalities are present or missing; never imply missing data was observed.	Must	Run with omitted modality.
R-06	Return class labels, scores, score type, model version, warnings and run status.	Must	API contract test.
R-07	Generate Grad-CAM only for supported image models and SHAP only for supported features.	Should	Known test input; inspect mapping.
R-08	Show explanation limitations; attention weights are not causal explanations.	Must	UI/content review.
R-09	When RAG is enabled, link each citation to retrieved source metadata and abstain if evidence is weak.	Must	Valid and empty-corpus tests.
R-10	Record run provenance and audit events without logging raw clinical inputs.	Must	Inspect logs and run record.
R-11	Allow export and deletion under a documented retention policy.	Must	Export and deletion tests.
R-12	Provide offline evaluation with confusion matrix, macro-F1 and applicable class metrics.	Must	Fixed-split metric test.

5. Developer design: modules and contracts

Module	Input → output	Key rule
Web UI	User actions → API requests and rendered status/results	No secrets; accessible text and clear errors.
API / domain	JSON + files → case/run/report resources	AuthZ and validation happen server-side.
Preprocessing	Validated artifact + model manifest → tensors/features	Versioned; fit transforms on training split only.
Model runner	Features + modality mask → class scores	Class order and score meaning are explicit.
XAI	Run + target output → heatmap/attributions	May fail independently from prediction.
RAG/report	Query + approved retrieved chunks → cited draft	Sources are untrusted text; citations checked.
Storage	Metadata + encrypted artifacts → protected persistence	Random object keys; controlled download.

6. Input and model rules

The paper describes four possible input types. Do not build all four before confirming the dataset actually contains them and that they can be aligned per subject/case.

Input	Possible paper-inspired handling	Developer must configure
Image	Resize/normalize for chosen model; training augmentation only.	Format, dimensions, channels, normalization, max bytes.
EEG/ECG signal	Filter, window, optionally create spectrogram.	Sampling rate, channels, units, window/overlap, filter, STFT.
Structured clinical data	Impute and scale values.	Feature names/order/units, missingness, ranges, train-fitted statistics.
Clinical text	Tokenize with biomedical language model.	Allowed fields, tokenizer/version, max tokens, truncation and privacy rules.

The paper's sample architecture uses ResNet-50, EfficientNet-B3, an MLP, BioBERT, shared 512-dimensional embeddings and attention fusion. Treat these as options—not fixed API assumptions. Put dimensions, labels, preprocessing, accepted modalities and missing-data behavior in a validated, versioned task/model manifest.

7. API sketch

REST/JSON is one reasonable choice. Prefix: /api/v1. Long-running work returns 202 Accepted plus a run ID; client polls the status endpoint.

Endpoint	Purpose
POST /api/v1/cases	Create research case.
POST /api/v1/cases/{id}/artifacts	Upload an input file.
POST /api/v1/cases/{id}/validate	Validate task compatibility.
POST /api/v1/cases/{id}/runs	Start inference.
GET /api/v1/runs/{id}	Read status/result.
GET /api/v1/runs/{id}/explanations	List XAI artifacts.
POST /api/v1/runs/{id}/report	Start optional cited report.
DELETE /api/v1/cases/{id}	Request case deletion.

Example request: POST /api/v1/cases/{id}/runs → {"model_version":"demo-v1","requested_outputs":["prediction","gradcam"]}.
Example response: {"run_id":"…","status":"QUEUED"}.

Use consistent errors, e.g. {"error":{"code":"SCHEMA_MISMATCH","message":"Input columns do not match this task.","retryable":false},"request_id":"…"}. Never send stack traces or filesystem paths.

8. Data and provenance

Record	Store at minimum
Case	Owner, task, pseudonymous reference, status, created/deleted times.
Artifact	Case ID, modality, checksum, detected type, schema version, validation status, private storage key.
Run	Model/task/preprocess versions, input artifact IDs, present modalities, result scores, warnings, timestamps.
Explanation	Run ID, method/version, target class, artifact or attribution data.
Report	Run ID, corpus/generator/template versions, report text, citation IDs and status.
Audit event	Actor, action, object ID, timestamp, outcome, request ID—no raw input.

Keep binaries in private storage. Store model and dataset versions, split manifest/checksum, code revision, labels, random seed and model checksum so experiments can be reproduced.

9. RAG report: simple rules

- Build an approved, versioned source collection; record source title, URL/identifier, date and usage/license status.
- Retrieve top-k passages using a pinned embedding/index version; keep source IDs with each passage.
- Treat retrieved text as reference data, never as instructions to the model.
- Validate that report citations refer to passages actually retrieved.
- If retrieval is empty or weak, say evidence was insufficient. Do not invent recommendations or citations.
- Do not generate prescriptions, dosage, treatment orders or emergency triage advice.

10. Security and privacy basics

- Prototype uses de-identified data only. Real clinical data require separate institutional, ethics, legal and security approval.
- Use TLS in deployment, encrypted storage, least privilege, short-lived download links and server-side authorization.
- Quarantine and validate uploads; cap size, parsing time and concurrency. Do not trust filename or browser MIME type.
- Do not log images, full text, signals, secrets or access tokens. Add rate limits and audit events.
- Define retention and backup-expiry policy; deletion should cover original and derived artifacts.

11. Testing: definition of done

Test group	Minimum checks
Unit	Validators, transforms, class ordering, metrics, citation checks, authorization helpers.
Integration	Upload-to-run; model loading; status transitions; deletion; XAI/RAG partial failures.
End-to-end	Create case → upload → run → review → optional report → export/delete.
ML evaluation	Patient-level split where IDs exist; no preprocessing leakage; report class-wise results and missing-modality performance.
Security	Cross-user access, malicious files, prompt injection, log redaction, rate limits.

Release checklist: ☐ one task has a documented data dictionary ☐ model manifest validates ☐ clean setup works ☐ negative upload tests pass ☐ run provenance recorded ☐ limitations visible ☐ no clinical claims made.

12. Client decisions — please check/complete

Decision	Recommended starting point	Select / fill in
First dataset and prediction task	One dataset/task for first milestone	☐ _____
Available modalities	Only those paired in the chosen dataset	☐ Image ☐ Signal ☐ Structured ☐ Text
Client technology preference	Python/PyTorch for ML; client chooses UI/API stack	☐ _____
Deployment	Local demo first; hosted deployment needs security decisions	☐ Local ☐ Hosted
RAG source and LLM	Off by default until source/model approved	☐ Off ☐ Approved: _____
Input limits / hardware	Set after choosing sample formats and machine	☐ _____
Retention period	Specify project and backup deletion policy	☐ _____
Evaluation goal	Educational baseline or reproduction attempt	☐ Baseline ☐ Reproduction

13. Build order

Step	Deliverable	Ready when…
1. Decide scope	Dataset, task, labels and permitted use	Data and label mapping reviewed.
2. Make a vertical slice	One modality through UI/API/model/evaluation	Demo works and tests pass.

3. Add fusion	Additional available modality + missing-data policy	Ablation results and provenance recorded.
4. Add XAI	Supported explanation view	Correctly linked to inference run.
5. Add RAG	Approved corpus and cited draft report	Citation and abstention tests pass.
6. Harden	Auth, deletion, audit, deployment docs	Acceptance checklist passes.

14. Paper-derived context and important caveat

The supplied paper proposes a unified multimodal CNN + attention + XAI + RAG pipeline and reports benchmark results such as 94.2% on ChestX-ray14. These are the paper's claims, not project guarantees. The project must report its own dataset version, split method, metrics, limitations and validation scope. This document is a simplified requirements baseline; it is not a medical device specification or proof of regulatory compliance.

Glossary

Term	Plain-language meaning
Modality	A kind of data, such as an image or signal.
Fusion	Combining model features from different data types.
Grad-CAM	A heatmap showing image regions associated with a model score.
SHAP	A method that estimates how input features relate to a model output.
RAG	Find source passages first, then use them to help draft a grounded summary.
Manifest	Versioned configuration defining a task, model, inputs and preprocessing.
Abstain	Say there is not enough evidence rather than guess.

Part II — Recommended technology stack

This stack follows the requested exclusions: no database, Docker, automated testing framework, or background-job system is included. The first release is a simpler single-process research prototype with synchronous inference and local private file storage.

Area	Recommended technology	Role / notes
Frontend	React + TypeScript + Vite	Browser-based case intake, uploads, result/explanation/report views.
UI styling	Tailwind CSS + shadcn/ui	Consistent, accessible components.
Backend API	Python + FastAPI + Pydantic	REST API, request validation, application logic and synchronous inference orchestration.
ML framework	PyTorch + Torchvision	CNN encoders, multimodal fusion and model training/inference.
Image processing	Pillow; MONAI if required	Image loading and medical-imaging workflows when needed.
Signal processing	NumPy + SciPy	EEG/ECG filtering, windows and signal transforms.
Tabular data	Pandas + scikit-learn	Structured feature preprocessing and evaluation utilities.
Explainability	Captum or pytorch-grad-cam; SHAP	Grad-CAM image visualizations and feature attributions.
File storage	Local private file storage	Uploaded inputs and generated explanation artifacts; keep outside public web directory.
RAG retrieval	Sentence Transformers + FAISS	Embeddings and local retrieval over an approved document collection; optional feature.
LLM	Approved hosted API or locally deployed model	Drafting evidence-linked summaries; keep disabled until provider/source approval.

Stack rules for the first version

- Run inference synchronously for small demo workloads; return a clear timeout/size error if the request exceeds configured limits.
- Store application metadata in versioned JSON/CSV files only if needed for the demo; do not build a database layer in this scope. Treat this as a prototype limitation, not a production recommendation.
- Do not add Docker, a job queue, or a test suite as dependencies in this requested stack. Manual verification and clear run instructions are still necessary.
- Keep the ML pipeline modular so a database, queue, or deployment platform can be added later without rewriting model code.
- Use only approved, de-identified research inputs; local storage must have access restrictions and a documented cleanup policy.

Part III — Codebase structure

Suggested lightweight monorepo structure. It intentionally omits database modules, Docker/infra, automated test folders, and background workers per your requested stack.

```
multimodal-medical-ai/
├── apps/
│   ├── api/
│   │   └── app/
│   │       ├── main.py                # FastAPI app and startup
│   │       ├── api/v1/
│   │       │   ├── cases.py           # Case create/read/delete endpoints
│   │       │   ├── artifacts.py       # Synchronous upload/validation endpoints
│   │       │   ├── runs.py            # Synchronous inference endpoint
│   │       │   └── reports.py         # Optional evidence-summary endpoint
│   │       ├── core/
│   │       │   ├── config.py          # Environment-backed settings
│   │       │   ├── errors.py          # Stable API error codes
```

```
│   │       │   └── security.py         # Auth and access policy hooks
│   │       ├── schemas/               # Pydantic request/response models
│   │       ├── services/              # Case, upload, orchestration logic
│   │       └── storage/               # Local private file-store adapter
│   └── web/
│       ├── src/
│       │   ├── pages/                # Dashboard, case, result pages
│       │   ├── components/           # Upload, score, explanation, citation UI
│       │   ├── api/                  # Typed API client
│       │   └── types/                # Frontend data contracts
│       └── package.json
├── ml/
│   ├── manifests/                    # Versioned task/model configs
│   ├── preprocessing/                # Image, signal, tabular, text transforms
│   ├── encoders/                     # Modality-specific encoder adapters
│   ├── fusion/                       # Modality masks, attention, classifier
│   ├── inference/                    # Model loading and prediction interface
│   └── explainability/               # Grad-CAM and SHAP adapters
├── rag/                              # Optional, feature-flagged
│   ├── ingestion/                    # Approved source import and chunking
│   ├── retrieval/                    # Embeddings and FAISS index
│   └── generation/                   # Prompting and citation validation
├── shared/
│   ├── constants.py                  # Shared enums and status values
│   └── errors.py                     # Shared error identifiers
├── scripts/
│   ├── prepare_data.py               # Dataset-specific preparation
│   ├── evaluate.py                   # Offline model evaluation
│   └── build_knowledge_index.py      # Optional approved-corpus indexing
├── docs/
│   ├── SRS.md                        # Requirements baseline
│   ├── DATA_DICTIONARY.md            # Input fields, units and labels
│   ├── MODEL_CARD.md                 # Intended use, limits, evaluation
│   └── RUNBOOK.md                    # Local setup and demo instructions
├── .env.example                      # Example config; no secrets
├── .gitignore
├── pyproject.toml                    # Python dependencies and tooling
└── README.md
```

Folder responsibilities

Folder	Responsibility
apps/api	HTTP contracts, validation, synchronous orchestration and local file access.
apps/web	User interaction and rendering; never stores model secrets.
ml	Task manifests, preprocessing, model encoders, fusion and XAI. Keep independent of the web/API framework.
rag	Optional retrieval/report feature, isolated so it can remain disabled.
shared	Small shared constants and error vocabulary.
scripts	Manual dataset preparation, evaluation and index building commands.
docs	Requirements, dataset/model documentation, and local operating instructions.

Suggested request flow

Browser → FastAPI route → validate request and file → save to private local storage → preprocess → load selected model manifest → run inference → optionally compute explanation and RAG summary → return result JSON. The synchronous design is appropriate only while dataset/input sizes and user concurrency remain small.

Project setup notes

- Python 3.11+ is a reasonable starting point. Install FastAPI, Uvicorn, Pydantic Settings, python-multipart and the selected ML packages through pyproject.toml.

- Frontend: Node.js LTS with React, TypeScript, Vite and the selected UI packages.
- Keep model weights, datasets, uploaded files and secrets out of Git unless redistribution is explicitly permitted.
- Do not expose the local storage directory as static/public content. Generate controlled download responses after access checks.
- When the app outgrows synchronous local operation, revisit the excluded database, queue and deployment infrastructure with the client.

Combined document note

This document combines the simplified visual SRS, recommended technology stack, and proposed repository structure. Choices are initial recommendations and should be confirmed against the client's chosen dataset, task, deployment environment, and institutional requirements. Medical AI outputs remain research-only and require qualified human review.